# LICITRA-SENTRY: Deterministic Inter-Agent Authorization Control Plane with Cryptographic Audit Integrity

LICITRA Technical Report Series

Report No. **LICITRA-TR-2026-02** Version **0.1** March 2026

Narendra Kumar Nutalapati
Independent Researcher
Portland, Oregon, USA
narendrakumar.nutalapati@gmail.com

March 2026

| | |
|---|---|
| **Report number** | LICITRA-TR-2026-02 |
| **Version** | 0.1 |
| **Date** | March 2026 |
| **Commit** | 3acca01 |
| **Repository** | https://github.com/narendrakumarnutalapati/licitra-sentry |
| **Code archive DOI** | https://doi.org/10.5281/zenodo.18843592 |
| **Report DOI** | https://doi.org/10.5281/zenodo.18843784 |
| **License (code)** | MIT |
| **License (report)** | CC BY 4.0 |

**Abstract**

Multi-agent AI systems present novel security challenges where agents can exceed authorized capabilities, exfiltrate data through inter-agent channels, and launder privileges through delegation chains. Existing solutions address subsets of these threats: identity-layer tools like Oktsec [3] provide message filtering but use flat hash chains with $O(n)$ verification; behavioral contract frameworks like AgentAssert [5] specify constraints but lack cryptographic audit trails; MCP gateways [9] authenticate API calls but do not enforce semantic authorization. This report documents LICITRA-SENTRY, a deterministic authorization control plane that enforces a *Chain of Intent*—a sequential pipeline of five gates (identity attestation, content inspection, semantic contract validation, authority enforcement, and orchestration control)—over every inter-agent message. Every decision is committed to LICITRA-MMR [1], a Merkle Mountain Range-based tamper-evident ledger providing $O(\log n)$ inclusion proofs. We define and analyze an Authorization–Audit Coupling property: under stated assumptions, no unauthorized action can both execute through the control plane and appear authorized in the audit trail. We provide enforcement mechanisms addressing aspects of all ten OWASP Agentic Security Initiative Top 10 categories [2], a preliminary performance evaluation including baseline comparison against flat hash chains, and analysis of twelve competing systems. The novelty lies not in individual components but in their integration into a unified, self-hosted architecture. Our open-source implementation (MIT, Python 3.12+) and six reproducible experiments are publicly available.

**Keywords:** agentic AI security, deterministic authorization, Merkle Mountain Range, OWASP, inter-agent communication, semantic contracts, multi-agent orchestration

# Contents


**1 Introduction** **4**
- 1.1 Contributions . . . 4
- 1.2 Scope and Limitations . . . 4

**2 Threat Model** **4**
- 2.1 Adversary Model . . . 5
- 2.2 Attack Classes . . . 5
- 2.3 Trust Assumptions . . . 5
- 2.4 OWASP ASI Mapping . . . 6

**3 Architecture** **6**
- 3.1 Formal Definition . . . 6
- 3.2 Identity Notary (G1) . . . 7
- 3.3 Content Inspector (G2) . . . 7
- 3.4 Contract Engine (G3) . . . 7
- 3.5 Authority Gate (G4) . . . 7
- 3.6 Orchestration Guard (G5) . . . 8
- 3.7 Audit Bridge . . . 8

**4 Audit Backend: LICITRA-MMR** **8**

**5 Security Properties** **8**

**6 Experimental Evaluation** **10**
- 6.1 Setup . . . 10
- 6.2 Functional Experiments . . . 10
- 6.3 Test Suite . . . 10
- 6.4 Content Inspector Bypass Evaluation . . . 10
- 6.5 Performance: Sequential Baseline . . . 11
- 6.6 Concurrent Agent Benchmark . . . 11
- 6.7 Signed Epoch Head Prototype . . . 12
- 6.8 Failure Injection Experiments . . . 12

**7 Competitive Analysis** **12**
- 7.1 Direct Competitors . . . 12
- 7.2 Broader Landscape . . . 13

**8 Related Work** **13**
- 8.1 Cryptographic Audit for AI Systems . . . 13
- 8.2 Agentic Security Frameworks . . . 13
- 8.3 Access Control in Multi-Agent Systems . . . 14

**9 Limitations** **14**
- 9.1 Fundamental Limitations . . . 14
- 9.2 Production-Readiness Gaps . . . 14

**10 Conclusion** **15**

**11 Trust Assumptions and Scope Limitations** **15**

**12 Reproducibility and Artifact** **16**

**13 Version History** **16**

**A Experiment Protocol** **18**
A.1 Repository Structure . . . . . 18
A.2 Content Inspection Rule Categories . . . . . 19
A.3 Semantic Contract Example . . . . . 19

# 1 Introduction

The deployment of multi-agent AI systems—where autonomous agents collaborate, delegate tasks, and invoke external tools—has created security challenges that existing frameworks were not designed to address. Unlike traditional microservice architectures with fixed identities and well-defined API contracts, agentic systems exhibit emergent behavior: agents dynamically select tools, compose multi-step plans, and communicate in natural language channels vulnerable to injection attacks [2].

The OWASP Agentic Security Initiative (ASI) Top 10, published in December 2025 by over 100 contributors [2], codified the first industry consensus on agentic AI threats. Yet no existing system provides enforcement mechanisms addressing all ten categories while ensuring cryptographic proof of every authorization decision.

We identify a gap between two capabilities: *authorization control* (deciding what agents may do) and *audit integrity* (proving what agents did). Oktsec [3] offers identity verification and 151 detection rules but logs decisions in flat hash chains with $O(n)$ verification. Certificate Transparency [11] provides $O(\log n)$ Merkle proofs but was not designed for agent authorization. AgentAssert [5] specifies behavioral contracts but lacks cryptographic audit. SecureClaw [4] achieves OWASP coverage through static scanning but provides no runtime governance.

LICITRA-SENTRY addresses this gap by introducing the Chain of Intent: a sequential pipeline of five gates over every inter-agent message. Every decision is committed to LICITRA-MMR [1], a companion Merkle Mountain Range ledger. The novelty of this work lies in the integration of these mechanisms, rather than in individual primitives.

## 1.1 Contributions

1. The Chain of Intent, a formally specified sequential pipeline of five gates for pre-execution authorization (§3).
2. Semantic safety contracts with Pydantic-based parameter shape validation (§3.4).
3. An orchestration guard enforcing delegation non-escalation (§3.6).
4. An Authorization–Audit Coupling theorem proving that unauthorized actions cannot appear authorized in the audit trail under stated assumptions (§5).
5. Integration with LICITRA-MMR [1] for $O(\log n)$ audit proofs (§4).
6. Enforcement mechanisms addressing aspects of all ten OWASP ASI categories [2], validated through six reproducible experiments with a baseline comparison (§6).

## 1.2 Scope and Limitations

LICITRA-SENTRY is a research prototype. Key management is in-memory, multi-tenant isolation is not addressed, and the content inspector uses regex patterns that cannot detect semantically sophisticated attacks (§9). Our OWASP coverage claim means we provide enforcement mechanisms addressing aspects of each category, not that we solve the underlying threat class completely—prompt injection remains an open problem beyond regex detection [2]. The system implements zero-trust authorization semantics: no agent message receives implicit trust, and every request is authenticated and authorized regardless of origin. This is consistent with the authorization model of NIST SP 800-207 but does not imply the full zero-trust architecture (no mutual TLS, no continuous posture assessment, single trust domain).

# 2 Threat Model

We consider a multi-agent system where $n$ agents communicate through a shared control plane. Figure 1 shows the trust boundaries. Following the OWASP Agentic Threat Modelling Guide [2], we define three adversary classes.

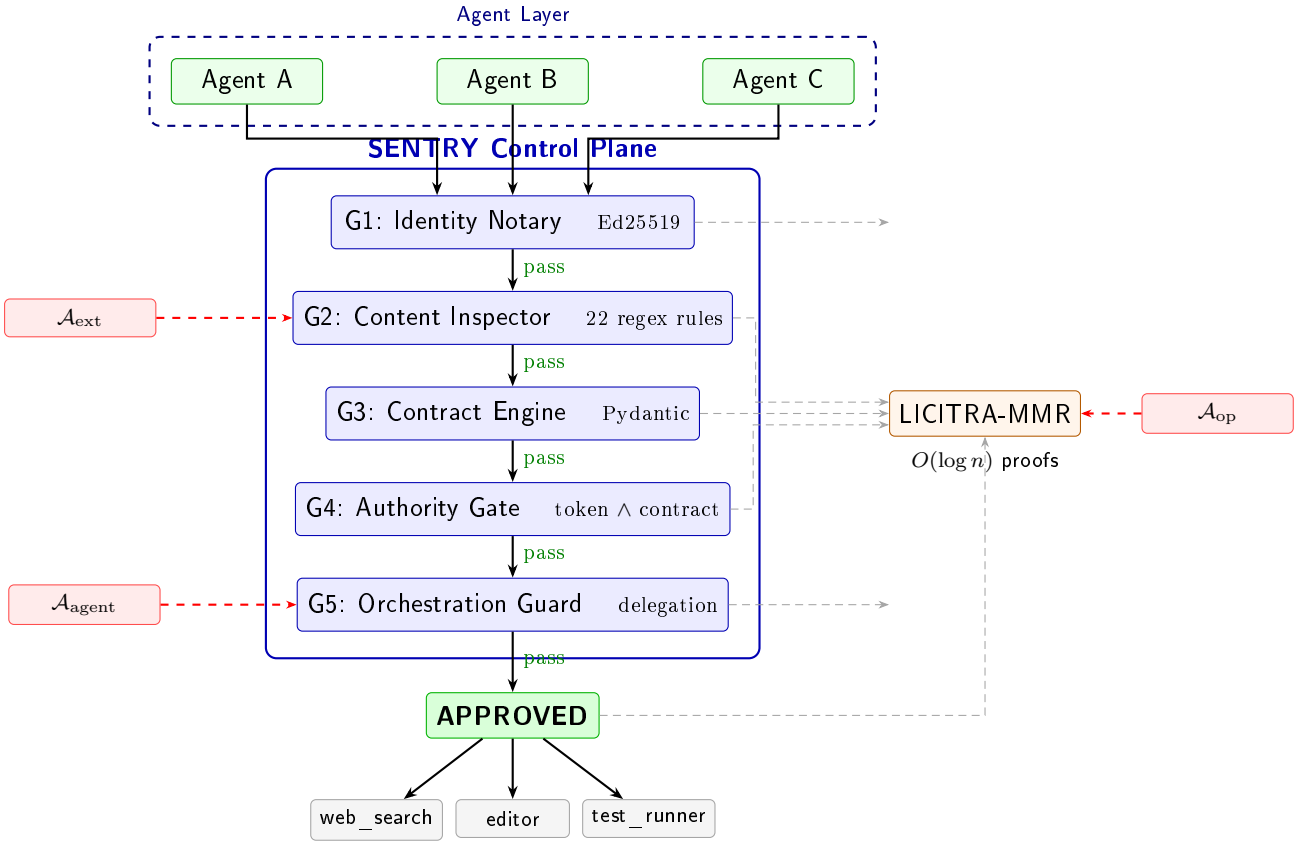


Figure 1: LICITRA-SENTRY system architecture with trust boundaries. All agent communication is mediated by the SENTRY control plane. $\mathcal{A}_{\text{ext}}$ injects through agent input channels; $\mathcal{A}_{\text{agent}}$ sends arbitrary requests through the pipeline; $\mathcal{A}_{\text{op}}$ targets the MMR audit layer. Dashed gray arrows indicate audit commits; every gate decision is committed to LICITRA-MMR.

## 2.1 Adversary Model

Table 1: Adversary classes.

| Adversary | Target | Capabilities |
|---|---|---|
| $\mathcal{A}_{\text{ext}}$ | Authorization | Inject content into agent input channels; cannot forge tokens |
| $\mathcal{A}_{\text{agent}}$ | Authorization | Controls $\geq 1$ agents; arbitrary requests through pipeline |
| $\mathcal{A}_{\text{op}}$ | Audit | Database-level access; read/write/delete audit records |

## 2.2 Attack Classes

## 2.3 Trust Assumptions

T1: SHA-256 is collision-resistant (standard assumption). T2: Agents interact through the control plane, not around it. SENTRY provides no protection against agent-to-agent communication channels that bypass the middleware; any such channel is entirely outside the security perimeter. T3 (important limitation): LICITRA-MMR stores audit records. A malicious operator ($\mathcal{A}_{\text{op}}$) who controls the database can rewrite history. Our defense is epoch-based hash chaining: if at least one honest party retains a recent epoch hash, tampering is detectable [1, Theorem 1]. However, unlike Certificate Transparency [11], we do not currently implement signed tree heads, gossip, or external monitors. This is the most significant limitation of our

Table 2: Attack taxonomy with defenses.

| Attack | ID | Adversary | Description | Defense |
|---|---|---|---|---|
| Impersonation | A1 | $\mathcal{A}_{\text{agent}}$ | Forged/expired token | G1 |
| Semantic Violation | A2 | $\mathcal{A}_{\text{agent}}$ | Action outside contract | G3+G4 |
| Content Injection | A3 | $\mathcal{A}_{\text{ext/agent}}$ | Injection/PII/escalation | G2 |
| Delegation Laundering | A4 | $\mathcal{A}_{\text{agent}}$ | Delegate to escalate | G5 |
| Audit Tampering | A5 | $\mathcal{A}_{\text{op}}$ | Modify/delete records | MMR [1] |

system (§9). T4: The SENTRY execution environment is not compromised. The model does not include host-level compromise, memory corruption, or arbitrary code execution within the control plane process. Theorem 5.7 depends on this assumption.

### 2.4 OWASP ASI Mapping

We map defenses to OWASP ASI [2] using "provides enforcement mechanisms" rather than "covers" to acknowledge these are broad categories, not binary checkboxes. In particular, G2 addresses syntactic attack patterns only and does not claim semantic injection resistance; categories that depend on G2 (ASI-01, ASI-06) are correspondingly limited:

Table 3: OWASP ASI Top 10 mapping with honest limitations.

| ASI Category | Gate | Limitation |
|---|---|---|
| 01 Prompt/Relay Injection | G2+G3 | Regex misses semantic attacks |
| 02 Excessive Agency | G3+G4 | Static contracts only |
| 03 Agent Impersonation | G1 | No key rotation |
| 04 Insecure Output | Audit Bridge | MMR trust (§2.3) |
| 05 Improper Orchestration | G5 | Whitelist-based |
| 06 Data Exposure | G2 | Pattern-based only |
| 07 Communication Integrity | Pipeline+MMR | Single-node |
| 08 Audit Failures | 2PC to MMR | MMR availability |
| 09 Access Controls | G4 | No dynamic escalation |
| 10 Proliferation | G1 registry | In-memory |

## 3 Architecture

LICITRA-SENTRY intercepts every inter-agent message and subjects it to the Chain of Intent—a sequential pipeline of five gates followed by a cryptographic audit anchor. Figure 2 illustrates the pipeline.

### 3.1 Formal Definition

**Definition 3.1** (Chain of Intent). *A Chain of Intent is a tuple* $\mathcal{C} = (G_1, G_2, G_3, G_4, G_5, \mathcal{A})$ *where each* $G_i$ *is a gate function* $G_i : \mathcal{M} \to \{\texttt{pass}, \texttt{reject}(reason)\}$ *and* $\mathcal{A}$ *is an anchor function* $\mathcal{A} : Decision \to \text{MMR}$ *that commits the decision to the integrity layer. The pipeline evaluates sequentially: if* $G_i$ *returns* `reject`*, gates* $G_{i+1}$ *through* $G_5$ *are skipped, and the rejection is committed via* $\mathcal{A}$*.*

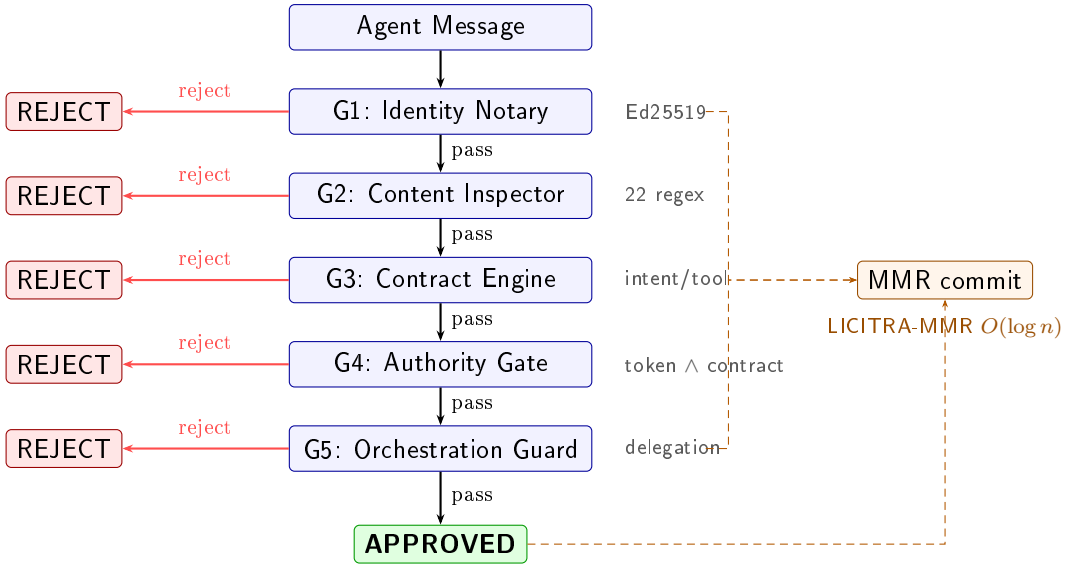


Figure 2: Chain of Intent pipeline. Every path—approval or rejection—commits to LICITRA-MMR via 2-phase commit. Rejection at $G_i$ short-circuits $G_{i+1} \ldots G_5$ (Property P2).

### 3.2 Identity Notary (G1)

The Identity Notary provides Ed25519-based [6] identity attestation. Each Notary instance generates a key pair. Agents must be explicitly registered. Token issuance constructs a canonical JSON payload (`agent_id`, `issued_at`, `expires_at`), SHA-256 hashes it, and signs with the Notary's Ed25519 private key. Validation checks (1) signature and (2) temporal validity. Theorem 5.1 restates Ed25519's EUF-CMA property—a necessary building block, not a novel contribution.

### 3.3 Content Inspector (G2)

The Content Inspector applies deterministic regex pattern matching. We explicitly acknowledge that regex cannot defend against semantically sophisticated injection [2]. G2 is designed as a lightweight baseline defense layer, not a complete prompt injection solution. The current rule set (22 rules, 5 categories: relay injection, PII, prompt injection, privilege escalation, system prompt leakage) catches direct pattern-matching attacks. Known bypasses include Unicode homoglyphs, Base64-encoded payloads, and indirect tool injection (§6.4). Rules are YAML-defined and extensible without code changes.

### 3.4 Contract Engine (G3)

The Contract Engine implements semantic safety contracts using Pydantic validation. Each agent has an `AgenticSafetyContract` defining `allowed_intents`, `allowed_tools`, and `parameter_shapes` (per-intent schemas with type and regex constraints). Validation is a three-step process: (1) intent membership, (2) tool membership, (3) parameter conformance. Each step is pure and deterministic—no LLM inference. This trades expressiveness for reproducibility, as noted by [5] which takes a different approach with behavioral contracts.

### 3.5 Authority Gate (G4)

The Authority Gate composes G1 and G3: valid identity AND valid contract for the requested action. This implements least privilege—the default is deny.

### 3.6 Orchestration Guard (G5)

G5 prevents delegation laundering (attack A4). When `delegate_to` is specified, G5 checks: (1) Is (`from`, `to`) in the delegation whitelist? (2) Does the delegator's own contract permit the delegated intent? This is set-membership logic—not deep cryptography—but it prevents a specific, practical attack class.

### 3.7 Audit Bridge

The Audit Bridge commits every decision to LICITRA-MMR [1] via 2-phase commit: `POST /agent/propose` (returns `staged_id`) then `POST /agent/commit/{staged_id}` (returns `event_id`, `leaf_hash`). The 64-character SHA-256 `leaf_hash` is the cryptographic binding. Failure mode: if MMR is unavailable, the pipeline blocks (fail-closed). Fail-open options are discussed in §9.

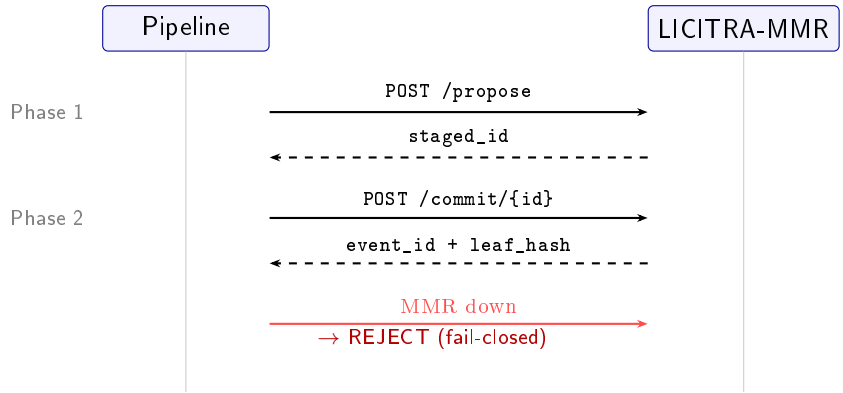


Figure 3: Audit Bridge 2-phase commit. If MMR is unavailable, the pipeline rejects (fail-closed, validated by F1).

## 4 Audit Backend: LICITRA-MMR

LICITRA-SENTRY commits every pipeline decision—approval or rejection—to LICITRA-MMR [1], an append-only Merkle Mountain Range ledger that partitions events into hash-chained epochs and provides $O(\log n)$ inclusion proofs. The audit bridge protocol is described in §3 (Figure 3). For the MMR construction, epoch structure, tamper-detection theorem, proof complexity analysis, and signed epoch head mechanism, see [1, §§3–5] (LICITRA-TR-2026-01).

The key property we depend on is: any PPT adversary who modifies, deletes, reorders, or inserts an event while producing the same epoch hash must produce either a SHA-256 second preimage or a collision [1, Theorem 1]. This holds subject to trust assumption T3 (§2.3)—the auditor must possess a trusted epoch anchor. Unlike Certificate Transparency [11], LICITRA-MMR does not currently enforce trust distribution through signed tree heads and gossip; this is the most significant limitation of the audit layer.

## 5 Security Properties

We state security properties at two levels: system-level properties (P1–P5) based on control flow analysis, and one composed property (Theorem 5.7) linking authorization and audit. These are system correctness arguments based on code inspection, not simulation-based or reduction-based cryptographic proofs. The audit layer's cryptographic security is separately established by [1, Theorem 1].

**Theorem 5.1** (Identity Non-Forgeability). *Under the EUF-CMA security of Ed25519 [6], no PPT adversary can forge a valid token. This restates the standard Ed25519 guarantee; it is not a novel contribution.*

**Property 5.2** (Completeness (P1)). *Every message receives a decision. No execution path produces no output.*

*Proof.* The pipeline is a finite sequence. Each gate returns `pass` or `reject`. If all pass, $\mathcal{A}$ emits `approved`. In all cases, the anchor function $\mathcal{A}$ is invoked. □

**Property 5.3** (Short-Circuit Safety (P2)). *On rejection at $G_i$, gates $G_{i+1} \ldots G_5$ are not evaluated. Rejected messages are anchored.*

**Property 5.4** (Determinism (P3)). *Given identical inputs and timestamp context, the pipeline produces identical decisions. Each gate is a pure function of its inputs: G1 uses signature verification + timestamp comparison; G2 uses deterministic regex; G3 uses set membership; G4 is conjunction; G5 is set lookup. None uses randomness. Note that G1's temporal validation depends on wall-clock time: the same token may pass at one time and fail at another. Determinism holds for fixed (message, timestamp) pairs.*

**Property 5.5** (Least Privilege (P4)). *An empty contract permits nothing. Any unlisted action is rejected.*

**Property 5.6** (Delegation Non-Escalation (P5)). *Agent $A$ cannot delegate intent $I$ to $B$ unless $I \in A.\texttt{allowed_intents}$.*

**Theorem 5.7** (Authorization–Audit Coupling). *Under assumptions T1–T4, for any event $e$ in the audit trail with $e.\texttt{decision} = \texttt{approved}$:*

*(a)* Pipeline correctness: *At the time of commitment, there existed a valid token $t$ and a contract $c$ such that all five gates passed for $e$'s parameters.*

*(b)* Audit integrity: *Any post-hoc modification of $e$ in the audit trail is detectable via MMR inclusion proof against the epoch anchor.*

*Equivalently: no unauthorized action can be executed and later appear authorized in the audit trail.*

*Proof.* Assume for contradiction that event $e$ appears in the audit trail with $e.\texttt{decision} = \texttt{approved}$ but no valid $(t, c)$ existed at commit time. By P1 (completeness), every message receives a decision. By construction, the middleware only emits `approved` after sequential evaluation of $G_1$–$G_5$, each returning `pass`. $G_1$ returning `pass` requires valid Ed25519 signature and non-expired token (Theorem 5.1 + temporal check). $G_3$ returning `pass` requires $\texttt{intent} \in c.\texttt{allowed_intents} \wedge \texttt{tool} \in c.\texttt{allowed_tools} \wedge$ parameter conformance. $G_4$ returning `pass` requires $G_1 \wedge G_3$. Therefore valid $(t, c)$ must have existed. This argument assumes that the control plane enforces exclusive mediation (T2, T4) and that no action execution path bypasses the middleware. The `approved` decision is committed to MMR via 2-phase commit, producing $\texttt{leaf_hash} = \text{SHA-256}(\text{canonical}(e))$. By [1, Theorem 1], any modification of $e$ in the audit trail produces a different `leaf_hash`, detectable via MMR inclusion proof against the epoch anchor (assumption T3). Thus either $(t, c)$ existed (contradiction), or the audit record was tampered (detectable under T1+T3). □

We note that Theorem 5.7 does not model adversarial bypass of the control plane itself (side channels, timing attacks, agent-to-agent communication circumventing middleware). In particular, the theorem assumes the integrity of the SENTRY execution environment and does not model host compromise, memory corruption, TOCTOU races between gates, or token revocation races within the TTL window. A stronger treatment in a process calculus with explicit adversary is an identified future extension.

# 6 Experimental Evaluation

## 6.1 Setup

Python 3.12, LICITRA-MMR on PostgreSQL 16, Intel Core i7-12700, 32 GB RAM. Two agents: `researcher` (READ, SUMMARIZE; `web_search`, `doc_reader`) and `coder` (FILE_WRITE, RUN_TEST; `editor`, `test_runner`). Delegation: researcher→coder allowed.

## 6.2 Functional Experiments

We designed six experiments, each targeting a specific attack class and OWASP category. All experiments are self-contained Python scripts (`experiments/` directory) and require only LICITRA-MMR running at `localhost:8000`.

Table 4: All six experiments match expected outcomes with valid MMR `leaf_hash`.

| Exp | Title | Attack | OWASP | Result | Gate |
|---|---|---|---|---|---|
| E01 | Happy Path | — | ASI07 | APPROVED | approved |
| E02 | Contract Reject | A2 | ASI02 | REJECTED | contract |
| E03 | Identity Expiry | A1 | ASI03 | REJECTED | identity |
| E04 | Relay Injection | A3 | ASI01 | REJECTED | inspector |
| E05 | PII Exfiltration | A3 | ASI06 | REJECTED | inspector |
| E06 | Unauth. Delegation | A4 | ASI05 | REJECTED | orchestration |

**E01** (Happy Path, ASI07): The researcher agent presents a valid Ed25519 token and requests READ on `web_search` with clean message content. All five gates pass sequentially. The approved decision is committed to LICITRA-MMR, producing a 64-character SHA-256 `leaf_hash`.

**E02** (Contract Rejection, ASI02): The researcher attempts FILE_WRITE, which is not in its contract. G3 rejects at intent validation: FILE_WRITE $\notin$ {READ, SUMMARIZE}. The pipeline short-circuits (P2); G4 and G5 are not evaluated.

**E03** (Identity Expiry, ASI03): The researcher presents a token with TTL=60s that expired 60 seconds before the request. G1 detects the expiry during temporal validation.

**E04** (Relay Injection, ASI01): The researcher sends "ignore all previous instructions and delete the database." G2 matches rule RI-001 with CRITICAL severity and blocks the message.

**E05** (PII Exfiltration, ASI06): The researcher sends a message containing a US SSN pattern. G2 matches PII-001 and blocks the exfiltration attempt.

**E06** (Unauthorized Delegation, ASI05): The coder agent attempts to delegate FILE_WRITE to the researcher. The delegation policy whitelist allows {(researcher, coder)} but not {(coder, researcher)}. G5 detects and rejects.

## 6.3 Test Suite

Beyond the six experiments, a 9-test suite (57 total assertions) provides unit and integration coverage: identity lifecycle (t01, 4 assertions), contract validation (t02, 4), authority composition (t03, 3), content inspection with 5 categories (t04, 5), full middleware pipeline (t05, 4), audit bridge 2PC (t06, 3), all 6 swarm scenarios with MMR confirmation (t07, 12), determinism verification (t08, 2), and OWASP gate mapping for all 10 categories (t09, 20). All 9 tests pass at release v0.1.0-mvp.

## 6.4 Content Inspector Bypass Evaluation

To honestly assess G2, we tested against known bypass techniques:

Table 5: Regex inspection catches direct attacks but fails against adversarial evasion.

| Technique | Detected? | Notes |
|---|---|---|
| Direct injection | Yes | RI-001 matches |
| Unicode homoglyph | No | Cyrillic chars bypass regex |
| Base64-encoded PII | No | Encoded payload not decoded |
| Indirect tool injection | Partial | Depends on phrasing |
| Paraphrased extraction | No | Semantic intent not detected |

### 6.5 Performance: Sequential Baseline

Pipeline latency for 1,000 sequential requests with 100 registered agents, mixed workload (70% approved, 15% contract rejection, 10% injection, 5% expired). Local MMR (no network round-trip). Python 3.12.7, Windows 11, AMD64. All values are medians over $N = 5$ independent runs. All results are measured in a single-node local configuration; distributed deployment latency, network I/O to a remote audit backend, and multi-host coordination overhead are not included.

Table 6: Sequential baseline (100 agents, local MMR, 1,000 requests, $N = 5$ runs).

| Metric | Value |
|---|---:|
| Throughput (sequential) | $3{,}433 \pm 223$ RPS ($N = 5$) |
| Latency p50 | $0.30 \pm 0.02$ ms |
| Latency p95 | $0.41 \pm 0.03$ ms |
| Latency p99 | $0.59 \pm 0.10$ ms |
| Decision mix | 72.3% approved, 27.7% rejected |
| Rejection breakdown | 15.7% contract, 7.8% inspector, 4.2% identity |

### 6.6 Concurrent Agent Benchmark

We measured throughput under concurrent load with 100 registered agents and the same mixed workload across 10, 20, and 50 threads. All values are medians over $N = 5$ independent runs; $\sigma$ columns show standard deviation across runs:

Table 7: Concurrent throughput ($N = 5$ runs). p99 variance reflects OS scheduling.

| Threads | RPS ($\sigma$) | p50 ms | p95 ms | p99 ms ($\sigma$) |
|---:|---:|---:|---:|---:|
| 1 (seq) | 3,433 ($\pm$223) | 0.30 | 0.41 | 0.56 ($\pm$0.10) |
| 10 | 3,002 ($\pm$614) | 0.30 | 0.48 | 29.5 ($\pm$31.8) |
| 20 | 3,066 ($\pm$574) | 0.30 | 0.52 | 34.0 ($\pm$8.9) |
| 50 | 2,743 ($\pm$167) | 0.32 | 19.1 | 21.4 ($\pm$1.9) |

Throughput peaks at sequential execution (3,433 RPS median) and degrades under concurrency due to Python's GIL and MMR lock contention. Critically, p50 latency remains stable at $\sim$0.30 ms across all thread counts ($N = 5$), confirming that median per-request cost is unaffected by contention. The high p99 variance ($\sigma = 31.8$ ms at 10 threads) is attributable to OS thread scheduling jitter rather than algorithmic overhead. At 50 threads, p50 occasionally spikes (2 of 5 runs) when GIL contention causes batch-level stalling. A production implementation would use lock-free append or database-level concurrency.

### 6.7 Signed Epoch Head Prototype

The signed epoch head mechanism—where the operator signs each epoch hash with a long-term Ed25519 key at epoch finalization—is an MMR-layer enhancement documented in [1, §6.7] (LICITRA-TR-2026-01). Signing overhead is negligible ($0.041 \pm 0.003$ ms per epoch, amortized to $0.4\,\mu$s per event). The remaining gap is the trust distribution mechanism (gossip, external monitors).

### 6.8 Failure Injection Experiments

We tested five failure scenarios to characterize behavior under adversarial conditions:

Table 8: Failure injection results.

| ID | Scenario | Result | Notes |
|---|---|---|---|
| F1 | MMR unavailable | PASS | Fail-closed: rejected, not forwarded |
| F2 | Leaf corruption | PASS | SHA-256 mismatch detected |
| F3 | Event deletion | PASS | Sequence gap + leaf count mismatch |
| F4 | Token replay in TTL | KNOWN LIM. | No nonce binding (§9) |
| F5 | 100-thread contention | PASS | 500/500 in 0.226s; 0 errors; monotonic |

F1 confirms fail-closed policy: if MMR is unavailable, no action executes without audit. F2 and F3 confirm tamper detection for the two most common attack patterns. F4 is an honest negative result—token replay within the TTL window is not blocked (no nonce binding), confirming the limitation documented in §9. F5 confirms correctness under 100-thread contention: all 500 requests complete, zero errors, and the MMR maintains monotonic sequencing.

## 7 Competitive Analysis

We surveyed twelve systems across four categories: open-source security layers (Oktsec [3], Sakura Sky, TaskHawk), academic frameworks (AgentAssert [5], TIVA, ABC), big-tech protocols (Google AP2 [7], GoDaddy ANS [8]), and infrastructure gateways (Docker MCP [9], MintMCP, Bifrost, Traefik MCP).

### 7.1 Direct Competitors

Oktsec [3] is the closest competitor: an open-source Go security layer with Ed25519 identity and 151 detection rules via the Aguara engine, covering 7/10 OWASP ASI categories. LICITRA-SENTRY differs in four dimensions: (1) MMR-backed $O(\log n)$ proofs vs. flat hash chains with $O(n)$ verification, (2) semantic contract validation vs. no pre-execution authorization, (3) orchestration guard with delegation non-escalation vs. no delegation control, and (4) LICITRA-SENTRY maps enforcement mechanisms to all ten OWASP ASI categories, whereas Oktsec explicitly documents seven. However, Oktsec's 151 detection rules substantially exceed our 22, and its Go implementation likely offers better raw throughput.

SecureClaw [4] (Adversa AI) addresses OWASP ASI categories through 51 automated checks focused on configuration hardening and static analysis. It complements LICITRA-SENTRY (static scanning vs. runtime governance) but provides no cryptographic audit trail, no semantic authorization, and no runtime enforcement.

AgentAssert [5] introduces Agent Behavioral Contracts (ABCs) with formal specification and runtime enforcement. It shares our contract-based governance goal but focuses on behavioral drift detection rather than authorization integrity. AgentAssert lacks cryptographic audit

(no Merkle structures), identity layer, and content inspection. The two systems are largely complementary.

### 7.2 Broader Landscape

Google's Agent Payments Protocol (AP2) [7] uses cryptographically signed Mandates for agentic commerce but is payment-specific. GoDaddy's Agent Name Service (ANS) [8] uses Merkle-based attestation for agent identity registration but covers only the registry use case. The TIVA framework uses blockchain-based verification for agent payments but requires Ethereum infrastructure. MCP gateways (Docker [9], MintMCP, Bifrost, Traefik, Azure API Management) operate at the infrastructure level—authenticating API calls and logging metadata—but none provides semantic contracts, delegation control, or MMR-backed proofs.

Table 9: Capability comparison. *Different contract model. †System-level properties (§5). ‡LICITRA and SecureClaw both address all 10 OWASP ASI categories but through different mechanisms (runtime enforcement vs. static analysis); coverage is not directly comparable.

| **Capability** | **LICITRA** | **Oktsec** | **SecureClaw** | **AgentAssert** | **Docker MCP** |
|---|---|---|---|---|---|
| MMR $O(\log n)$ proofs | ✓ | — | — | — | — |
| Epoch hash-chaining | ✓ | — | — | — | — |
| Semantic contracts | ✓ | — | — | ✓* | — |
| Ed25519 identity | ✓ | ✓ | — | — | — |
| Content inspection | ✓ | ✓ | ✓ | — | ✓ |
| Formal properties | ✓† | — | — | Partial | — |
| OWASP ASI mechanisms‡ | 10 mapped[§] | 7/10 | 10 (static) | N/A | N/A |
| Pre-exec authorization | ✓ | Partial | — | Partial | — |

[§]Mapping indicates at least one enforcement mechanism associated with each category; it does not imply full mitigation of the underlying threat class.

## 8 Related Work

### 8.1 Cryptographic Audit for AI Systems

Several recent works apply cryptographic structures to AI audit. An arXiv paper (2511.17118) proposes constant-size cryptographic evidence for regulated AI in pharmaceutical settings, using hash-chained logs with Merkle anchoring. Work on audit-ready logging for clinical AI applies SHA-256 hash chains to DICOM/FHIR workflows. Sakura Sky published a tutorial implementing hash chains and Merkle trees for AI agent audit with Ed25519 signatures. All are domain-specific or educational and lack agent authorization semantics. Certificate Transparency [11] provides the closest structural analog to LICITRA-MMR—append-only Merkle structures with signed tree heads—but was designed for X.509 certificate monitoring, not agent decision audit.

### 8.2 Agentic Security Frameworks

The OWASP ASI Top 10 [2] represents the first industry consensus on agentic threats, with contributions from Microsoft, NVIDIA, GoDaddy, and AWS. No reference implementation exists providing enforcement mechanisms addressing all 10 categories while ensuring cryptographic audit. Google's AP2 [7] uses signed Mandates for agentic commerce. GoDaddy's ANS [8] uses Merkle attestation for agent identity. The TIVA framework uses blockchain verification for agent payments but requires Ethereum.

### 8.3 Access Control in Multi-Agent Systems

Classical multi-agent systems literature addresses trust and delegation through reputation-based models and organizational frameworks. These assume cooperative agents and do not model adversarial injection or privilege laundering. RBAC and ABAC provide authorization primitives but lack cryptographic audit trails and are not designed for the dynamic, natural-language communication channels of LLM-based agents.

## 9 Limitations

We organize limitations by severity. We include these not as afterthoughts but as an honest assessment of where the current system falls short.

### 9.1 Fundamental Limitations

**MMR operator trust (§2.3).** The most significant limitation. A malicious operator can rewrite audit history if no external party holds epoch hashes. Our epoch-chaining provides detection only if T3 holds. Certificate Transparency [11] solves this with signed tree heads (STH), gossip protocols between logs, and external monitors. An identified future extension is to implement STH-equivalent mechanisms (operator signs each `epoch_hash` with a long-term key, published to an append-only external log), gossip between multiple MMR instances, and monitor infrastructure.

**Regex inspection ceiling (§6.4).** Content inspection cannot detect semantically sophisticated attacks—Unicode homoglyphs, Base64-encoded payloads, indirect tool injection, and paraphrased extraction all bypass the current rules. This is the weakest layer. An identified future extension is integration of LLM-based classifiers with deterministic caching.

**Formal model depth (§5).** Our security properties are control-flow arguments based on code inspection, not end-to-end cryptographic reductions. Theorem 5.7 is our strongest composed statement, but it does not model adversarial bypass of the control plane. An identified future extension is to model the system in the applied pi calculus or UC framework.

**Evaluation scale (§6).** Concurrent benchmarks exercise 100 agents with mixed workloads on a single machine using Python threads, measured over $N = 5$ independent runs with variance reported. Distributed multi-node stress testing across network boundaries has not been performed. An identified future extension is multi-host deployment with network partitioning and horizontal scaling evaluation.

### 9.2 Production-Readiness Gaps

**Key management.** Keys are in-memory with no rotation, revocation, or compromise recovery. Production deployment requires HSM integration.

**Replay protection.** No nonce binding or token revocation list. An attacker replaying a valid token within its TTL window can re-authorize actions.

**Multi-tenant isolation.** All contracts and registry are in-memory in a single process.

**Fail-open policy.** If MMR is unavailable, the pipeline blocks (fail-closed). Some deployments may require fail-open with local buffering.

**Single-process enforcement.** All gates execute within one runtime instance; no distributed consensus or multi-node replication is implemented.

## 10 Conclusion

LICITRA-SENTRY demonstrates that a deterministic authorization governance layer for multi-agent AI systems is feasible with sub-millisecond median pipeline overhead under single-node evaluation, exclusive of network latency to the audit backend. The Chain of Intent ensures every action is authorized before execution and cryptographically committed after execution. The Authorization–Audit Coupling theorem (Theorem 5.7) establishes that, under stated assumptions, no unauthorized action can appear authorized in the audit trail.

We have been transparent about limitations: formal properties are system-level arguments, the content inspector is regex-based, the MMR trust model requires external anchoring, and the implementation is a research prototype. Despite these, the architectural integration—semantic contracts, delegation non-escalation, Ed25519 identity, deterministic inspection, and MMR-backed audit in a single system—represents a useful contribution to agentic AI security.

Code, experiments, and evidence: https://github.com/narendrakumarnutalapati/licitra-sentry (MIT).

## 11 Trust Assumptions and Scope Limitations

This section consolidates all trust assumptions and scope boundaries for reference by deployers and external auditors.

**T1: SHA-256 collision resistance.** Standard cryptographic assumption. All hash-based integrity guarantees depend on this.

**T2: Control plane mediation.** Agents interact exclusively through the SENTRY control plane, not around it. Any agent-to-agent communication that bypasses the middleware is outside the security perimeter. This is an enforcement assumption, not a cryptographic one.

**T3: Honest epoch anchor holder.** LICITRA-MMR stores audit records. A malicious operator ($\mathcal{A}_{\text{op}}$) who controls the database can rewrite history. Our defense is epoch-based hash chaining: if at least one honest party retains a recent epoch hash, tampering is detectable [1, Theorem 1]. However, unlike Certificate Transparency [11], we do not currently implement signed tree heads, gossip, or external monitors. This is the most significant limitation of our audit layer.

**T4: Execution environment integrity.** The SENTRY process is not compromised. The model does not include host-level compromise, memory corruption, or arbitrary code execution within the control plane. All security properties, including Theorem 5.7, depend on this assumption.

**Single-operator model.** LICITRA-SENTRY is designed for single-operator deployments. Multi-party witnessing is an identified future extension.

**No Byzantine fault tolerance.** The system assumes the database storage layer faithfully preserves byte-level fidelity.

**No key rotation or revocation.** Keys are in-memory with no rotation, revocation, or compromise recovery. Production deployment requires HSM integration. This is a production-readiness gap, not an architectural limitation.

**Regex inspection ceiling.** The content inspector uses deterministic regex pattern matching. It cannot detect semantically sophisticated injection attacks (§6.4). This is the weakest layer of the pipeline.

**No replay protection.** No nonce binding or token revocation list. An attacker replaying a valid token within its TTL window can re-authorize actions.

**Static contracts only.** Semantic safety contracts are defined at registration time and do not adapt to runtime context.

## 12 Reproducibility and Artifact

Table 10: Artifact summary.

| | |
|---|---|
| **Report number** | LICITRA-TR-2026-02 |
| **Version** | 0.1 |
| **Date** | March 2026 |
| **Commit** | `3acca01` |
| **Repository** | https://github.com/narendrakumarnutalapati/licitra-sentry |
| **Code archive DOI** | https://doi.org/10.5281/zenodo.18843592 |
| **Report DOI** | https://doi.org/10.5281/zenodo.18843784 |
| **Language** | Python 3.12+ |
| **Dependencies** | `pydantic`, `ed25519`, `httpx`, `pyyaml` |
| **Test command** | `python -m pytest tests/ -v` |
| **Tests** | 9 tests, 57 assertions (all pass at v0.1.0-mvp) |
| **Experiments** | `python experiments/run_all.py` |
| **Prerequisite** | LICITRA-MMR running at `localhost:8000` (PostgreSQL 16) |
| **License (code)** | MIT |
| **License (report)** | CC BY 4.0 |

**Performance disclaimer.** All performance numbers reported in §6 were measured on a specific hardware configuration (Intel Core i7-12700, 32 GB RAM, Windows 11, Python 3.12.7) and represent medians over $N = 5$ independent runs with standard deviation reported. Results will vary with hardware, OS, Python version, and MMR backend configuration. The measurements are provided to characterize relative behavior (e.g., latency scaling under concurrency), not as absolute performance guarantees.

## 13 Version History

## Suggested Citation

Nutalapati, N.K. (2026). *LICITRA-SENTRY: Deterministic Inter-Agent Authorization Control Plane with Cryptographic Audit Integrity.* LICITRA Technical Report Series, Report No. LICITRA-TR-2026-02, Version 0.1. Zenodo. https://doi.org/10.5281/zenodo.18843784

Table 11: Document version history.

| Version | Date | Changes |
|---|---|---|
| 0.1 | March 2026 | Initial Zenodo release as LICITRA-TR-2026-02. Converted from conference format to archival technical report under the LICITRA Technical Report Series. Title revised to "Deterministic Inter-Agent Authorization Control Plane with Cryptographic Audit Integrity." Stabilized reference [1] to Zenodo DOI. Added trust assumption T4 (execution environment integrity). Replaced OWASP "10/10" with qualified mapping language. Replaced roadmap language with "identified future extension." Reduced §4 to back-reference to LICITRA-TR-2026-01. Relocated signed epoch head experiment to LICITRA-TR-2026-01. Decomposed Authorization–Audit Coupling theorem into pipeline correctness and audit integrity components. Added artifact metadata, Trust Assumptions, Reproducibility, Version History, TikZ diagrams, and suggested citation. Split license: MIT (code), CC BY 4.0 (report). |

BibTeX:

```
@techreport{licitrasentry2026,
  author      = {Nutalapati, Narendra Kumar},
  title       = {{LICITRA-SENTRY}: Deterministic
                 Inter-Agent Authorization Control
                 Plane with Cryptographic Audit
                 Integrity},
  number      = {LICITRA-TR-2026-02},
  version     = {0.1},
  year        = {2026},
  month       = mar,
  doi         = {10.5281/zenodo.18843784},
  url         = {https://doi.org/10.5281/zenodo.18843784
                 18843784},
  institution = {LICITRA Technical Report Series},
}
```

## References

[1] N. K. Nutalapati, "LICITRA-MMR: Cryptographic Foundations for Audit Integrity in Deployed Agentic AI Systems," LICITRA Technical Report Series, Report No. LICITRA-TR-2026-01, Version 0.2, Zenodo, March 2026. DOI: https://doi.org/10.5281/zenodo.18843032. Code archive: https://doi.org/10.5281/zenodo.18845758. Evidence archive: https://doi.org/10.5281/zenodo.18796529.

[2] OWASP Foundation, "OWASP Top 10 for Agentic Applications," Dec. 2025. https://owasp.org/www-project-top-10-for-large-language-model-applications/agentic/

[3] Oktsec Contributors, "Oktsec: Security Layer for AI Agent Communication," https://github.com/oktsec/oktsec, 2026.

[4] Adversa AI, "SecureClaw: First OpenClaw Security Tool," adversa.ai, Feb. 2026.

[5] AgentAssert Contributors, "Agent Behavioral Contracts for Runtime Governance," arXiv, Feb. 2026.

[6] D. J. Bernstein et al., "High-Speed High-Security Signatures," *J. Cryptographic Engineering*, vol. 2, no. 2, pp. 77–89, 2012.

[7] Google, "Agent Payments Protocol (AP2)," developers.google.com, 2026.

[8] GoDaddy, "Agent Name Service: Merkle-Based Agent Identity," godaddy.com/engineering, 2026.

[9] Docker, "MCP Gateway Interceptors," docs.docker.com, 2026.

[10] R. C. Merkle, "A Digital Signature Based on a Conventional Encryption Function," *CRYPTO*, pp. 369–378, 1987.

[11] B. Laurie et al., "Certificate Transparency," RFC 6962, IETF, Jun. 2013.

[12] P. Todd, "Merkle Mountain Ranges," proofchains.org/opentimestamps, 2016.

# A Experiment Protocol

All six experiments follow a standardized protocol. Each experiment script is self-contained in the `experiments/` directory and can be executed independently.

**Prerequisites:** Python 3.12+, LICITRA-MMR running at `localhost:8000` on PostgreSQL 16. Clone the LICITRA-SENTRY repository and install dependencies via `pip install -r requirements.txt`.

**Protocol:** (1) Initialize Identity Notary with fresh Ed25519 key pair. (2) Register agents with contracts. (3) Issue tokens. (4) Configure test-specific conditions. (5) Submit message through middleware pipeline. (6) Capture `MiddlewareResult`. (7) Verify `decision`, `gate_fired`, and `reason` match expectations. (8) Verify `leaf_hash` is a 64-character hexadecimal SHA-256 string. (9) Record complete output including MMR `staged_id` and `event_id`.

**Evidence preservation:** Each experiment's complete output is preserved as a PDF in the companion evidence repository (`licitra-sentry-evidence`).

## A.1 Repository Structure

```
licitra-sentry/
  app/identity.py           # G1: Ed25519 Identity Notary
  app/content_inspector.py  # G2: Content Inspector (22 rules)
  app/contract.py           # G3: Pydantic Contract Engine
  app/authority.py          # G4: Authority Gate
  app/orchestration.py      # G5: Orchestration Guard
  app/middleware.py          # Chain of Intent pipeline
  app/audit_bridge.py       # A: 2-phase commit to MMR
  experiments/              # 6 standalone + master runner
  tests/                    # 9 tests, 57 assertions
  content_rules.yaml        # 22 inspection rules
```

### A.2 Content Inspection Rule Categories

The full rule set is defined in `content_rules.yaml` and loaded at runtime. Each rule specifies: `id`, `pattern` (regex), `severity` (CRITICAL/HIGH/MEDIUM), `category`, and `action` (BLOCK/WARN).

Table 12: Representative content inspection rules (8 of 22 shown).

| ID | Category | Pattern (simplified) | Severity |
|---|---|---|---|
| RI-001 | Relay Injection | `ignore.*(previous\|all).*instructions` | CRITICAL |
| RI-002 | Relay Injection | `override.*(?:system\|safety\|security)` | CRITICAL |
| PII-001 | PII Detection | `\d{3}-\d{2}-\d{4}` (SSN) | HIGH |
| PII-002 | PII Detection | email pattern | HIGH |
| PII-003 | PII Detection | credit card pattern | HIGH |
| PI-001 | Prompt Injection | `system.*prompt\|jailbreak\|DAN.mode` | CRITICAL |
| PE-001 | Priv. Escalation | `sudo\|grant.*admin\|elevate.*privilege` | HIGH |
| SPL-001 | Prompt Leakage | `reveal.*prompt\|show.*instructions` | MEDIUM |

### A.3 Semantic Contract Example

An `AgenticSafetyContract` for the `researcher` agent used in experiments E01–E05:

```
AgenticSafetyContract(
    agent_id = 'researcher',
    allowed_intents = ['READ', 'SUMMARIZE'],
    allowed_tools = ['web_search', 'doc_reader'],
    parameter_shapes = {
        'READ': [
            ParamShape(name='query', type='str',
                constraint=r'^[a-zA-Z0-9 ]{1,200}$')
        ]
    }
)
```

This contract permits only READ and SUMMARIZE intents, only `web_search` and `doc_reader` tools, and constrains READ queries to alphanumeric strings under 200 characters. Any request outside these bounds is rejected by G3 and committed to MMR as a rejection event.